RepairDesk Location Features — Detailed Implementation Tasks (App + Backend)

Related: Wireframes (screen numbers refer to it) · Phase plan
Scope: Flutter app (`frontend/`) and Spring Boot backend (`backend/`). Admin website tasks are listed briefly at the end.
Status: Ready for development — 17 October 2026

Task IDs: `BE` = backend · `APP` = Flutter app · `WEB` = admin website · `QA` = testing.
Estimates are in developer-days for one developer who knows the codebase.
Every task is done only when the code is reviewed, has tests where stated, builds cleanly (`mvn verify` , `flutter analyze`), and is deployed to the dev environment on Azure.

1. Starting point (what exists today)

Area	Today	What this means for the work
Google Maps	Android, server and web keys created and tested. Server key is stored as the Azure secret `google-maps-server-key` (env `GOOGLE_MAPS_SERVER_KEY`); Android key is injected into the manifest from `local.properties`	Phase 0 is done, apart from daily quotas and the budget alert
Customer address	Requests and jobs store **no** service address; only the shop address exists in `business_settings`	Address + pin must be added first (BE-1.1)
Job actions	`POST /jobs/{id}/start` and `/complete` take no location; `JobService` copies request fields onto the job at approval	Location becomes an optional body field; address is copied the same way
Scheduling	No `@EnableScheduling` , no scheduled jobs	Needed for auto end-of-day, cleanup and ETA refresh (BE-1.5)
Outgoing HTTP	Backend calls no external services yet	New Google Maps client (BE-1.4)
Notifications	In-app inbox only (`notifications` table). No push notifications	Customer ETA push needs Firebase Cloud Messaging (Phase 6)
Database tests	H2 in PostgreSQL mode, schema from JPA (`create-drop`)	Store coordinates as plain `double precision` ; do distance maths in Java, not PostGIS
App navigation	Role-based home in `dashboard_screen.dart` (`_TechnicianDashboard` , `_StaffDashboard`) with screens pushed from tiles	New screens are added as tiles / pushed routes in the same style
Last migration	`V11__staff_assisted_intake.sql`	New migrations start at `V12`

2. Design decisions used in this plan

These use the defaults proposed earlier. Changing one changes only settings values, not the design.

Topic	Value
Shop geofence radius	150 m (setting)
Start day outside the shop	Not allowed unless Admin switches it on
Tracking interval while on duty	5 minutes, and also every 200 m moved (setting)
Working hours / auto end of day	9:00 AM – 7:00 PM (setting)
"Far from customer address" warning	More than 300 m from the pin (setting)

Topic	Value
Allowance rates	₹5 / km two-wheeler, ₹10 / km four-wheeler (setting, stored in paise)
Location history kept	90 days (setting), then deleted nightly
Coordinates	Latitude/longitude as `double precision`, accuracy in metres, all times in UTC (`timestamptz`)
Weak GPS never blocks work	A job action always succeeds; location is attached when available and flagged when poor
Fake GPS	Points from mock-location apps are accepted but flagged (`mocked = true`) and shown to Staff

3. Data model (new and changed tables)

Migration	Table / change	Key columns
`V12__service_address.sql`	`repair_requests`, `jobs`: add service address	`service_address_line`, `service_landmark`, `service_city`, `service_pincode`, `service_lat`, `service_lng`, `service_pin_source` (`GEOCODED` / `MANUAL_PIN` / `CHECK_IN`)
`V13__location_settings.sql`	`business_settings`: add shop pin. New singleton `location_settings`. `users`: add vehicle type	`shop_lat`, `shop_lng`, `geofence_radius_m`; feature switches, `ping_interval_minutes`, `ping_distance_m`, `day_auto_end_time`, `allow_start_outside_geofence`, `far_from_address_m`, `rate_two_wheeler_paise`, `rate_four_wheeler_paise`, `retention_days`, `privacy_notice_version`; `users.vehicle_type`
`V14__maps_cache_usage.sql`	`geocode_cache`, `maps_api_usage`	cache: `address_hash` (PK), `normalized_address`, `lat`, `lng`, `formatted_address`, `created_at`; usage: `day`, `api`, `call_count` (PK `day, api`)
`V15__job_visits.sql`	`job_visits`	`job_id`, `technician_id`, `visit_type` (`ON_MY_WAY` / `CHECK_IN` / `CHECK_OUT`), `lat`, `lng`, `accuracy_m`, `captured_at`, `distance_from_pin_m`, `low_accuracy`, `far_from_address`, `mocked`
`V16__attendance_days.sql`	`attendance_days`	`technician_id`, `work_date` (unique together), `start_at`, `start_lat/lng`, `start_inside_geofence`, `end_at`, `end_lat/lng`, `end_inside_geofence`, `end_reason` (`MANUAL` / `AUTO`)
`V17__location_points.sql`	`location_points`	`technician_id`, `captured_at`, `lat`, `lng`, `accuracy_m`, `speed_mps`, `battery_pct`, `mocked`, `client_point_id` (unique per technician, makes uploads safe to retry); index on `(technician_id, captured_at)`
`V18__travel_claims.sql`	`travel_days`, `travel_claims`	day: `technician_id`, `work_date`, `distance_m`, `legs_json`, `computed_at`; claim: `technician_id`, `period_from`, `period_to`, `distance_m`, `vehicle_type`, `rate_paise`, `amount_paise`, `status` (`SUBMITTED` / `APPROVED` / `REJECTED`), `decided_by`, `decided_at`, `note`
`V19__eta_push.sql`	`eta_sessions`, `device_tokens`	session: `job_id`, `technician_id`, `public_token` (random, unique), `started_at`, `ended_at`, `last_eta_seconds`, `last_distance_m`, `last_eta_at`; tokens: `user_id`, `fcm_token`, `platform`, `updated_at`
`V20__location_consents.sql`	`location_consents`	`user_id`, `notice_version`, `accepted_at`, `app_version`

All new tables follow the existing pattern: `uuid` primary key with `gen_random_uuid()`, `created_at`, foreign keys named `fk_…`, check constraints named `…_check`.

4. API endpoints (new and changed)

All paths are under `/api/v1`. The location body used everywhere is:
`{ "lat": 19.1197, "lng": 72.8468, "accuracyM": 12.5, "capturedAt": "2026-10-17T09:31:00Z", "mocked": false }`

Method and path	Roles	Purpose	Task
`GET /location/config`	All signed-in	Feature switches and intervals the app needs (no rates)	BE-1.3
`GET` / `PUT /settings/location`	Admin	Full location settings, shop pin, radius, rates	BE-1.3
`POST /geocode`	Staff, Admin, Customer	Address → pin (cached, counted)	BE-1.4
`PUT /jobs/{id}/service-location`	Staff, Admin	Correct the customer address or pin	BE-1.2
`POST /jobs/{id}/start` *(changed)*	Technician, Staff, Admin	Optional body `{ location }` = check-in	BE-2.2
`POST /jobs/{id}/complete` *(changed)*	Technician, Staff, Admin	Adds optional `location` = check-out	BE-2.2
`GET /jobs/{id}/visits`	Assigned technician, Staff, Admin	Visit record	BE-2.3
`POST /attendance/start`, `POST /attendance/end`	Technician	Start / end day with location	BE-3.2
`GET /attendance/today`	Technician	Duty card state	BE-3.2
`GET /attendance?from&to&technicianId`	Staff, Admin	Attendance report	BE-3.4
`GET /attendance/export.csv?from&to`	Staff, Admin	Excel-compatible export	BE-3.4
`POST /location/points`	Technician	Batch upload (up to 500 points)	BE-4.2
`GET /location/live`	Staff, Admin	Last position of every on-duty technician	BE-4.3
`GET /location/history?technicianId&date`	Staff, Admin; technician for self	Route for one day	BE-4.3
`GET /travel/days?from&to`	Technician (self), Staff, Admin	Daily distance and legs	BE-5.2
`POST /travel/claims`, `GET /travel/claims?status&month`	Technician / Staff, Admin	Submit and list claims	BE-5.3
`POST /travel/claims/{id}/approve` · `/reject` · `POST /travel/claims/bulk-approve`	Staff, Admin	Decide claims	BE-5.3
`GET /travel/claims/export.csv?month`	Staff, Admin	Monthly payout sheet	BE-5.3
`POST /jobs/{id}/on-my-way`	Assigned technician	Start customer ETA session	BE-6.1
`GET /public/track/{token}`	**Public** (no login)	Customer tracking page data	BE-6.3
`POST /devices/push-token`	All signed-in	Register phone for push	BE-6.2

Method and path	Roles	Purpose	Task
`POST /me/location-consent`, `GET /me/location-history?date`	Technician	Privacy acceptance and own history	BE-7.1

Phase 1 — Foundations (address, settings, maps service, permissions)

BACKEND

BE-1.1 Migration V12: service address on requests and jobs — 1 day

- **Files:** `db/migration/V12__service_address.sql`; `RepairRequest.java`, `Job.java`.
- Add the seven `service_*` columns (all nullable, so existing rows stay valid). Check constraint on `service_pin_source`; PIN code is 6 digits.
- New embeddable `ServiceLocation` (`common/domain/ServiceLocation.java`) used by both entities, so the code is shared.
- **Done when:** Flyway migrates the Azure dev database; existing requests and jobs load unchanged.

BE-1.2 Address in create, update, approve and detail — 2 days

- **Files:** `CreateRequestDto`, `UpdateRequestDto`, `RepairRequestResponse`, `RepairRequestService`; `JobResponse`, `JobSummary`, `JobService.approve`, `JobController`.
- Accept an optional `serviceLocation` object when a request is created or updated (Staff create-request and the customer's own request).
- If an address is given without a pin, geocode it through `MapsService` (BE-1.4) and set `pinSource = GEOCODED`; a dragged pin is `MANUAL_PIN`.
- In `JobService.approve`, copy the service location onto the job with the other copied fields.
- New `PUT /jobs/{id}/service-location` (Staff, Admin) with an audit entry `SERVICE_LOCATION_CHANGED`.
- Return `serviceLocation` in job and request detail; return only city/landmark in list summaries.
- **Done when:** unit tests cover create-with-address, create-with-pin, approve copies the address, and technician cannot change the pin.
- **Depends on:** BE-1.1, BE-1.4.

BE-1.3 Location settings and shop pin — 2 days

- **Files:** `V13__location_settings.sql`; new `feature/location/domain/LocationSettings.java`, `LocationSettingsRepository`, `LocationSettingsService`, `LocationSettingsController`; `BusinessSettings.java` (shop pin, radius); `User.java` (`vehicleType`).
- `GET/PUT /settings/location` for Admin with validation (radius 50–1000 m, interval 1–30 min, rates ≥ 0, retention 7–365 days).
- `GET /location/config` for any signed-in user, returning only switches, intervals and shop pin, so the app can adapt without a new release.
- Seed one row with the defaults from section 2; all feature switches start **off**.
- **Done when:** settings save and reload; non-admins get 403 on `PUT`; the app config endpoint never exposes rates.

BE-1.4 Google Maps client with cache, counter and fallback — 3 days

- **Files:** new `feature/maps/` — `GoogleMapsClient` (Spring `RestClient`), `MapsService`, `GeoMath` (haversine distance, point-in-radius), `GeocodeCache` entity + repository, `MapsUsage` entity + repository; `V14__maps_cache_usage.sql`; `AppProperties` (new `app.maps.server-key`, `app.maps.daily-limit-geocode`, `app.maps.daily-limit-routes`); `application.yml` (`server-key: ${GOOGLE_MAPS_SERVER_KEY:}`).
- `geocode(address)`: normalise (lowercase, trim, collapse spaces, append "India"), look up the cache by SHA-256 hash, otherwise call Geocoding with `region=in`, store and return.

- `route(from, to)` : Routes API `computeRoutes` , `TWO_WHEELER` or `DRIVE` , field mask `routes.distanceMeters,routes.duration` (keeps the cheaper billing tier).
- Count every call in `maps_api_usage` . Stop at 2,000 per API per day (below Google's 2,250 quota) and fall back to straight-line distance × 1.3, marked `estimated = true` .
- Timeouts of 5 seconds; never let a Maps failure fail a job action.
- If the key is empty (local development or tests), use the fallback only.
- `POST /geocode` endpoint, limited to 30 calls per user per hour.
- **Done when:** unit tests for `GeoMath` against known distances (Andheri–Bandra ≈ 7 km straight line); tests with a mocked client for cache hit, limit reached and Google error; one manual call on Azure dev confirms the secret is read.

BE-1.5 Turn on scheduling — 0.5 day

- **Files:** `RepairdeskApplication.java` (`@EnableScheduling`), new `config/SchedulingConfig.java` (thread pool of 2, `Asia/Kolkata` zone for cron).
- Because the Container App can scale to zero replicas, scheduled tasks must also run on demand: each task checks "has today's run happened?" in a small `scheduled_runs` row and catches up on the next request after a cold start.
- **Done when:** a sample task logs on schedule locally and catches up after a restart.

APP

APP-1.1 Packages, permissions and Android setup — 1 day

- **Files:** `pubspec.yaml` ; `android/app/src/main/AndroidManifest.xml` ; `android/app/build.gradle.kts` .
- Add `google_maps_flutter` , `geolocator` , `permission_handler` , `url_launcher` , `device_info_plus` , `android_intent_plus` .
- Manifest: `ACCESS_FINE_LOCATION` , `ACCESS_COARSE_LOCATION` , `POST_NOTIFICATIONS` now; background and foreground-service permissions are added in Phase 4 only, so earlier releases don't trigger Google's background-location rules.
- Confirm `minSdk` is at least 21 and set `compileSdk` to what `google_maps_flutter` needs.
- **Done when:** release APK builds, and a map renders on a test screen with the Android key.

APP-1.2 Location core service — 2 days

- **Files:** new `lib/core/location/location_fix.dart` (model + JSON), `location_service.dart` (Riverpod provider), `location_permission_service.dart` , `location_config_repository.dart` (`GET /location/config` , cached for the session).
- `bestFix({Duration timeout = 20s})` : listens to the position stream, returns the most accurate fix within the timeout, stops early at ≤ 20 m accuracy; returns `null` if location is off or denied. Never throws to the UI.
- Expose permission state as one enum: `granted` , `approximateOnly` , `whileInUseOnly` , `denied` , `deniedForever` , `serviceOff` .
- Copy `Position.isMocked` into `LocationFix.mocked` .
- **Done when:** unit tests for the "best fix" selection using a fake position stream.

APP-1.3 Permission onboarding and banners (Screens 1A, 1B) — 3 days

- **Files:** new `lib/features/location_onboarding/presentation/location_intro_screen.dart` , `battery_help_sheet.dart` ; `shared/widgets/location_banner.dart` ; `dashboard_screen.dart` (show once after login for technicians).
- Explanation screen first (1A), then the Android prompt (1B). Asked after login, not at install.
- Banners on the technician home: location off (button opens location settings), approximate only, "only this time" chosen.
- Battery help sheet with steps per brand (Xiaomi/Redmi, Realme, Oppo, Vivo, Samsung), detected with `device_info_plus` ; opens the right settings page with `android_intent_plus` where possible.
- Show only for technicians, and only when at least one location feature is switched on in `/location/config` .
- **Done when:** tested on the emulator for each permission answer, including "Don't allow" twice (permanently denied → opens app settings).

APP-1.4 Map widgets and address entry — 3 days

- **Files:** new `shared/widgets/map_preview.dart` (small static map with a pin, tap to open full screen), `shared/widgets/pin_picker_screen.dart` (drag pin, "use my location", returns lat/lng); `features/requests/presentation/new_request_screen.dart` (address fields + "Find on map"); `requests_repository.dart`, `jobs_repository.dart` (`serviceLocation` in models).
- Address fields: address line, landmark, city, 6-digit PIN code. "Find on map" calls `POST /geocode` and opens the pin picker at the result.
- Show the address and map preview on request detail and job detail.
- **Done when:** a customer and a Staff user can create a request with an address and a pin, and it appears on the job after approval.

Phase 2 — Geotagged check-in and check-out (feature 1)

BACKEND

BE-2.1 Migration V15 and visit domain — 1 day

- **Files:** `V15__job_visits.sql`; new `feature/visit/domain/JobVisit.java`, `VisitType.java`, `JobVisitRepository`, `dto/JobVisitResponse.java`, `dto/LocationFixDto.java` (shared location body with validation: lat −90..90, lng −180..180, accuracy 0..5000, time not more than 24 h old or 5 min in the future).
- **Done when:** entity tests pass on H2 and the migration runs on Azure dev.

BE-2.2 Location on Start Repair and Mark Complete — 2 days

- **Files:** `JobController` (`start` accepts an optional `StartRequest(LocationFixDto location)`; `CompleteRequest` gains `location`); `JobService.start`, `JobService.complete`; new `VisitService`.
- Keep old clients working: an empty body or no `location` must behave exactly as today.
- `VisitService.record(job, technician, type, fix)`: distance from the job pin with `GeoMath`; `low_accuracy` if accuracy > 100 m; `far_from_address` if distance > setting; copy `mocked`.
- If the job has no pin and the check-in accuracy is ≤ 50 m, save the check-in as the job pin with `pinSource = CHECK_IN` (feature 2.5 in the phase plan).
- If `far_from_address` or `mocked`, send an in-app notification to Staff/Admin: "RD-… checked in 1.2 km from the customer address".
- Audit entries `CHECKED_IN` / `CHECKED_OUT` with the distance.
- **Done when:** tests for no location, near, far, low accuracy, mocked, pin saved from check-in, and old-style empty body.
- **Depends on:** BE-1.2, BE-1.3, BE-2.1.

BE-2.3 Visits in job detail — 0.5 day

- **Files:** `JobController` (`GET /jobs/{id}/visits`), `JobResponse` (add `lastCheckIn`, `lastCheckOut` summary).
- Same access rule as job detail (assigned technician, Staff, Admin).
- **Done when:** controller test checks that another technician gets 403.

APP

APP-2.1 Capture location in the Start Repair and Mark Complete dialogs — 2 days

- **Files:** `features/jobs/presentation/job_detail_screen.dart` (`_run`, the "Start Repair?" dialog, the complete dialog); `jobs_repository.dart` (`transition` accepts `LocationFix?`).
- When the dialog opens, start `bestFix()` in the background; show "Getting your location… ±15 m" under the buttons.
- On confirm, wait up to the remaining part of 20 seconds, then send with or without a location. Weak or no GPS shows a note but never blocks.
- If permission is missing, offer "Turn on location" once, then continue without it.

- **Done when:** tested on the emulator with a simulated location, location off, and permission denied.

APP-2.2 Visit section on Job Detail (Screen 3) — 2 days

- **Files:** new `features/jobs/presentation/widgets/visit_section.dart`; `job_detail_screen.dart`; `jobs_repository.dart` (`visits(jobId)`).
- Before check-in: customer address, map preview, distance from the technician's current position, **Navigate** button (opens Google Maps with `google.navigation:q=lat,lng&mode=l` for two-wheeler, falls back to a web link).
- After completion: Visit record — check-in and check-out times, time on site, distance from address, and warning chips (far, low accuracy, mocked).
- **Done when:** matches Screen 3 in the wireframes on a 6-inch phone and a small 5-inch phone.

APP-2.3 Visit record for Staff and Admin in the app (Screen 6) — 1 day

- **Files:** `job_detail_screen.dart` (Staff/Admin view reuses `visit_section.dart` in read-only mode, with "View on map").
- **Done when:** Staff see the far-from-address warning on a job checked in from far away.

Phase 3 — Attendance by geofence (feature 2)

BACKEND

BE-3.1 Migration V16 and attendance domain — 0.5 day

- **Files:** `V16__attendance_days.sql`; new `feature/attendance/domain/AttendanceDay.java`, `AttendanceRepository`, `dto/AttendanceDayResponse.java`.
- `work_date` is the India date (`Asia/Kolkata`), not the UTC date.

BE-3.2 Start day and End day — 2 days

- **Files:** new `AttendanceService`, `AttendanceController`.
- `POST /attendance/start`: requires a location. Inside the shop radius → start. Outside → 409 with `OUTSIDE_GEOFENCE` and the distance, unless `allow_start_outside_geofence` is on (then start and flag it). Accuracy worse than the radius → 409 `LOW_ACCURACY` asking to retry outdoors. A second start on the same day returns the existing day.
- `POST /attendance/end`: location optional, records inside/outside.
- `GET /attendance/today`: state for the duty card (not started / on duty since 9:12 / ended).
- **Done when:** tests for inside, outside, outside-allowed, low accuracy, double start, end without start.
- **Depends on:** BE-1.3 (shop pin and radius).

BE-3.3 Automatic end of day — 1 day

- **Files:** new `AttendanceAutoEndTask` (`@Scheduled`, every 15 minutes).
- Ends open days after the working-hours end time with `end_reason = AUTO`, and sends the technician a notification.
- Also stops any open ETA session (Phase 6) for that technician.
- **Done when:** test with a fixed clock (`java.time.Clock` bean) covers before, at and after the end time.
- **Depends on:** BE-1.5.

BE-3.4 Attendance report and CSV export — 1.5 days

- **Files:** `AttendanceController` (`GET /attendance`, `GET /attendance/export.csv`), `AttendanceReportService`.
- One row per technician per day: start, end, hours, started inside/outside, auto-ended, jobs completed that day, distance (filled after Phase 5).
- CSV in UTF-8 with BOM so Excel opens it correctly; dates as `dd-MM-yyyy`, times in India time.
- **Done when:** export opens correctly in Excel with ₹ and Hindi names intact.

APP

APP-3.1 Duty card on the technician home (Screen 2) — 2 days

- **Files:** new `features/attendance/data/attendance_repository.dart`, `presentation/duty_card.dart`; `dashboard_screen.dart` (`_TechnicianDashboard`).
- States: not started (Start day button), checking location, outside shop area (shows distance; button disabled with explanation), on duty (since time, End day), ended.
- Start day uses `bestFix()` with a 20-second limit and shows the accuracy.
- Hidden when the attendance switch is off in `/location/config`.
- **Done when:** all states shown on the emulator by moving the simulated location in and out of the radius.

APP-3.2 Attendance report for Staff in the app — 1.5 days

- **Files:** new `features/attendance/presentation/attendance_report_screen.dart`; `dashboard_screen.dart` (new tile for Staff/Admin).
- Date range picker, per-technician list with flags; export is web-only (link to the admin site).
- **Done when:** Staff can view today's and last week's attendance.

Phase 4 — On-duty tracking (feature 3)

BACKEND

BE-4.1 Migration V17 and points domain — 0.5 day

- **Files:** `V17__location_points.sql`; new `feature/location/points/LocationPoint.java`, `LocationPointRepository` (batch insert using `saveAll` with JDBC batching enabled: `hibernate.jdbc.batch_size: 100`).
- Not in `feature/tracking`: that package already exists for **customer repair tracking**.

BE-4.2 Batch upload endpoint — 2 days

- **Files:** new `LocationPointController` (`POST /location/points`), `LocationPointService`.
- Accept up to 500 points; reject if the technician has no open attendance day covering the point time (protects privacy: no tracking off duty).
- Ignore duplicates by `(technician_id, client_point_id)`, so the app can safely resend after a network error.
- Drop impossible points (speed over 150 km/h compared with the previous point) but store them in a count for diagnostics.
- Returns `{ accepted, duplicates, rejected }`.
- **Done when:** tests for a normal batch, a resend, off-duty points and impossible jumps; load test of 50 technicians × 12 points per hour on the B1ms database stays under 200 ms per request.

BE-4.3 Live map and history — 2 days

- **Files:** `LocationPointController` (`GET /location/live`, `GET /location/history`).
- Live: for each on-duty technician, last point, time since last point, battery, current job (if `IN_PROGRESS`), and a `stale` flag after 15 minutes.
- History: points for one technician and one day, simplified to at most 1,000 points (Douglas–Peucker, 10 m tolerance) for drawing.
- Technicians may only read their own history.
- **Done when:** tests for access rules and simplification.

BE-4.4 Nightly cleanup — 0.5 day

- **Files:** new `LocationRetentionTask` (`@Scheduled`, 02:30 India time): delete `location_points` and `job_visits` coordinates older than `retention_days`, in chunks of 10,000 rows.

- **Done when:** test with a fixed clock deletes only old rows.

BE-4.5 Sort technicians by distance in Assign — 1 day (optional)

- **Files:** `UserManagementController` / `UserService` (`GET /users?role=TECHNICIAN&nearJobId=…`): adds `distanceM` from the technician's last live point to the job pin, sorted ascending; off-duty technicians at the end.

APP

APP-4.1 Background tracking service — 5 days

- **Files:** `AndroidManifest.xml` (add `ACCESS_BACKGROUND_LOCATION` , `FOREGROUND_SERVICE` , `FOREGROUND_SERVICE_LOCATION` , `WAKE_LOCK`); new `lib/core/location/duty_tracker.dart` .
- Use `geolocator` 's Android foreground-service mode (`AndroidSettings` with `foregroundNotificationConfig`) so Android keeps the app alive and shows the "Sharing location while on duty" notification. If testing on budget phones shows gaps, switch to `flutter_background_service` .
- Start on Start day, stop on End day, on auto end (checked from `/attendance/today` on every app resume), and on sign-out.
- Interval and distance filter come from `/location/config` .
- Background permission: after Start day, explain why, then open the "Allow all the time" settings page (Android 11+ does not show this as a dialog).
- After a phone restart, show a notification "Tap to resume duty tracking" instead of restarting silently.
- **Done when:** on the emulator, points keep arriving with the app swiped away and the screen off for 30 minutes.

APP-4.2 Offline queue and upload — 3 days

- **Files:** add `sqflite` , `path` , `connectivity_plus` , `battery_plus` to `pubspec.yaml` ; new `lib/core/location/point_queue.dart` , `point_uploader.dart` .
- Every point is saved to a local SQLite table first with a UUID `client_point_id` and battery level.
- Upload every 2 minutes or when 50 points are waiting, in batches of up to 500; delete after the server confirms; back off when offline (1, 2, 4… up to 15 minutes).
- Keep at most 7 days of unsent points (about 2,000), deleting the oldest first.
- **Done when:** airplane mode for 1 hour, then reconnect, uploads every point exactly once.

APP-4.3 Live technician map for Staff (Screen 5) — 3 days

- **Files:** new `features/live_map/data/live_map_repository.dart` , `presentation/live_map_screen.dart` ; `dashboard_screen.dart` (tile for Staff/Admin).
- Google map with a marker per technician (initials, colour by status), grey when stale; refresh every 60 seconds while the screen is open; list below the map; tap → day route (history) drawn as a polyline.
- **Done when:** matches Screen 5; refresh stops when the screen is closed.

APP-4.4 Field test on budget phones — 3 days (QA)

- Phones: Redmi, Realme, Vivo, Oppo, Samsung M/A series, with each brand's battery settings left at default and then fixed with the help sheet.
- Measure per day: gaps over 15 minutes, battery used, points uploaded, behaviour after restart and after the app is swiped away.
- **Done when:** a short report is written and the brand help steps (APP-1.3) are updated.

Phase 5 — Distance and travel allowance (feature 4)

BACKEND

BE-5.1 Daily distance calculation — 3 days

- **Files:** new `feature/travel/service/DistanceCalculator.java`, `TravelDayTask`, `TravelDay` entity, `V18__travel_claims.sql`.
- For each technician day: drop points with accuracy > 50 m, drop jumps (speed > 120 km/h), ignore movement under 25 m between consecutive points (GPS jitter while parked), sum the haversine distance.
- Split into legs at each job check-in and check-out, so each leg shows "Shop → RD-...", "RD-... → RD-...".
- Optional accuracy boost: for legs with big gaps (over 15 minutes without points), use one Routes API call for that leg; counted in the daily limit.
- Run nightly at 23:30 India time for days that ended, and on demand when a technician opens the travel screen.
- **Done when:** unit tests with recorded sample routes (a straight ride, a stop with jitter, a GPS jump, a gap) produce distances within 5% of the expected value.

BE-5.2 Travel days endpoint — 0.5 day

- **Files:** `TravelController` (`GET /travel/days`).

BE-5.3 Claims: submit, approve, export — 2 days

- **Files:** `TravelClaim` entity, `TravelClaimService`, `TravelController`.
- Submit for a date range: amount = distance (km, 1 decimal) × rate for the technician's vehicle type, rounded to the rupee. A day can be in only one non-rejected claim.
- Approve, reject with a reason, bulk approve; notify the technician.
- Monthly CSV export for payroll: technician, days, km, rate, amount, status, approved by.
- **Done when:** tests for amount calculation, overlapping claims, and permissions.
- **Depends on:** BE-5.1. Rates confirmed by the business.

APP

APP-5.1 Travel & allowance screen and Submit claim (Screen 4) — 3 days

- **Files:** new `features/travel/data/travel_repository.dart`, `presentation/travel_screen.dart`, `presentation/submit_claim_sheet.dart`; `dashboard_screen.dart` (tile for technicians).
- Week/month selector, daily km with legs, estimated amount, claim status chips; Submit claim for the selected range.
- **Done when:** matches Screen 4; amounts match the backend exactly (the app only displays them, it never calculates them).

APP-5.2 Approve claims for Staff — 1 day

- **Files:** `features/travel/presentation/claims_review_screen.dart`.
- List of submitted claims with approve / reject and select-all approve.

Phase 6 — Customer live ETA (feature 5)

BACKEND

BE-6.1 On my way and ETA sessions — 2 days

- **Files:** `V19__eta_push.sql`; new `feature/eta/` — `EtaSession` entity, `EtaService`, `EtaController`, `EtaRefreshTask`.
- `POST /jobs/{id}/on-my-way` (assigned technician, job `ASSIGNED` or `SCHEDULED`, on duty, job has a pin): create a session with a 32-character random token, record an `ON_MY_WAY` visit, calculate the first ETA.

- Refresh the ETA from the technician's latest point at most every 2 minutes, and only while someone is viewing (the public page or app asks); otherwise use the last value. This keeps Routes calls low.
- End the session on check-in, cancel, end of day, or after 3 hours.
- **Done when:** tests for permissions, refresh throttling and every way a session ends.

BE-6.2 Push notifications (Firebase Cloud Messaging) — 2 days

- **Files:** add `com.google.firebase:firebase-admin` to `pom.xml`; new `feature/push/` — `DeviceToken` entity, `PushService`, `DeviceTokenController`; Azure secret `firebase-service-account` (JSON) → env `FIREBASE_CREDENTIALS`.
- Send "Ravi is on the way — about 18 min" to the customer at On my way, and "Ravi has arrived" at check-in.
- Also used for Staff alerts from BE-2.2 and the technician auto-end notice from BE-3.3.
- If push is not configured, fall back to the in-app inbox only.
- **Done when:** a test phone receives the push from Azure dev.

BE-6.3 Public tracking endpoint and SMS — 2 days

- **Files:** `SecurityConfig` (permit `GET /public/track/**`); `PublicTrackingController`; new `feature/sms/SmsService` (provider adapter, e.g. MSG91), secrets `sms-api-key`, `sms-sender-id`, `sms-template-id`.
- Public data only: technician first name, shop name, ETA, distance band, technician position rounded to about 100 m, status. No customer address or phone in the response. Returns 410 when the session has ended.
- Limit 60 requests per token per hour.
- SMS with the link at On my way, only if the customer has no push token. Sending is switched off until DLT registration is approved.
- **Done when:** the link works without login and shows "Visit finished" after check-in.

APP

APP-6.1 Push setup in the app — 1.5 days

- **Files:** add `firebase_core`, `firebase_messaging`, `flutter_local_notifications`; `android/app/google-services.json` (git-ignored, like the Maps key); `main.dart`; new `lib/core/push/push_service.dart`.
- Ask for notification permission after login, register the token with `POST /devices/push-token`, refresh on change; tap opens the job or repair.
- **Done when:** customer, technician and Staff phones receive test pushes.

APP-6.2 "On my way" button for technicians — 1 day

- **Files:** `job_detail_screen.dart`, `visit_section.dart`, `jobs_repository.dart`.
- Shown for assigned jobs with a pin while on duty; confirms "Notify the customer that you are on the way?"; then shows "Customer notified · ETA 18 min".

APP-6.3 Customer tracking screen (Screen 7) — 2 days

- **Files:** `features/customer_home/presentation/repair_detail_screen.dart`, new `live_eta_card.dart`; `customer_home_repository.dart`.
- Map with the technician marker and the customer pin, ETA, technician name and call button; refresh every 30 seconds while open; "Arrived" state after check-in.

WEB-6.4 Public tracking web page — 2 days

- A public route `/track/:token` in the admin site (same domain, so the existing web key restriction already covers it), mobile-first, no login, polling every 30 seconds.

Phase 7 — Privacy, release and documentation

BE-7.1 Consent and own history — 1 day

- **Files:** `V20__location_consents.sql`; `LocationConsentController` (`POST /me/location-consent`, `GET /me/location-history`).
- Attendance start and point upload return 403 `CONSENT_REQUIRED` until the technician has accepted the current notice version.

APP-7.1 Privacy notice and acceptance — 1.5 days

- **Files:** new `features/location_onboarding/presentation/privacy_notice_screen.dart`; shown before Screen 1A; text in English and Hindi.
- Explains what is collected, why, only during duty hours, who can see it, and how long it is kept (DPDP Act 2023).

APP-7.2 My location history — 1 day

- **Files:** `features/account/presentation/account_screen.dart` (new "My location history" entry), `my_location_history_screen.dart` (date picker, route on a map).

QA-7.3 End-to-end test pass and release — 2 days

- Full flow on Azure dev with two technician phones: start day → on my way → check-in → complete → end day → next morning distance → claim → approval → export.
- Release APK with the Android key from the build machine; update the Play Store background-location declaration if published there.

DOC-7.4 Manuals — 2 days

- Update the Technician, Staff and Admin manuals with the new screens and regenerate the PDFs.

5. Admin website tasks (short list)

ID	Task	Est.
WEB-1.1	Add `@angular/google-maps`; load the Maps script with `environment.googleMapsApiKey`	0.5
WEB-1.2	Address + pin on Create Request and Request/Job detail	2
WEB-1.3	Location settings page (Screen 8): shop pin, radius, switches, intervals, rates	2
WEB-2.1	Visit record and far-from-address warning on job detail	1
WEB-3.1	Attendance report with CSV export	2
WEB-4.1	Live technician map and day route	3
WEB-5.1	Claims review, bulk approve, monthly export	2
WEB-6.4	Public tracking page (listed in Phase 6)	2

6. Effort summary

Phase	Backend	App	Web	QA / Docs	Total days
1 Foundations	8.5	9	4.5	—	22
2 Check-in / check-out	3.5	5	1	—	9.5
3 Attendance	5	3.5	2	—	10.5
4 On-duty tracking	6 (incl. optional 1)	11	3	3	23

Phase	Backend	App	Web	QA / Docs	Total days
5 Travel allowance	5.5	4	2	—	11.5
6 Customer ETA	6	4.5	2	—	12.5
7 Privacy and release	1	2.5	—	4	7.5
Total	**35.5**	**39.5**	**14.5**	**7**	**96.5**

About 19–20 working weeks for one developer, or about 10–11 weeks for two (one backend + web, one app). Phases 1–3 (check-in/out and attendance) can go live after about 8 weeks with one developer.

7. Suggested order of work

Each step ends with a release to technicians.

1. **Step 1:** BE-1.1 to BE-1.5 alongside APP-1.1 to APP-1.4, then WEB-1.x. Release: address and pin on jobs.
2. **Step 2:** Phase 2. Release: geotagged check-in/out.
3. **Step 3:** Phase 3 + BE-7.1 / APP-7.1 (consent is needed before any duty tracking). Release: attendance.
4. **Step 4:** Phase 4, including the field test. Release: on-duty tracking.
5. **Step 5:** Phase 5. Release: travel allowance.
6. **Step 6:** Phase 6. Start DLT registration during Step 4 so SMS is ready. Release: customer ETA.
7. **Step 7:** remaining Phase 7, manuals, hardening.

8. Still needed from the business

Item	Needed by
Daily quotas and ₹500 budget alert in Google Cloud	Before Phase 1 goes live
Shop location (exact pin) and radius	Phase 3
Allowance rates and who approves claims	Phase 5
Each technician's vehicle type	Phase 5
Firebase project (same Google account)	Phase 6
SMS provider account and DLT registration of sender ID and template (1–2 weeks)	Phase 6
Privacy notice text approved (English and Hindi)	Phase 3
Proper release signing key for the app (replaces the debug key; update the Android key's SHA-1 in Google Cloud)	Before the first release to technicians